TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

METODOLOGÍA DE SISTEMAS II

Trabajo Práctico: Flujos de trabajo colaborativo

Parte 1: Estrategias de Branching (Modelos de Ramificación)

1. Análisis de Modelos: Explique las diferencias fundamentales entre GitFlow y Trunk-based Development considerando los siguientes factores de gestión humana:

- La estructura de comunicación del equipo (jerárquica vs. sincronizada).

GitFlow es jerárquico y estructurado (ideal para controles formales), mientras que Trunk-based requiere una comunicación y sincronización total y constante entre desarrolladores.

- El costo de la "Deuda de Integración" (aislamiento prolongado de ramas).

En GitFlow las ramas de funcionalidad pueden aislarse por semanas, acumulando alta deuda y un costoso Merge Hell. En Trunk-based se integra diariamente, manteniendo la deuda al mínimo.

- La complejidad de los conflictos de fusión (Merge Hell).

Alta en GitFlow debido al aislamiento prolongado; mínima en Trunk-based gracias a la integración constante.

2. Caso de Estudio: Selección de Estrategia: Una empresa de software médico está desarrollando un sistema crítico que requiere auditorías estrictas y solo lanza actualizaciones cada tres meses. El equipo es grande y jerárquico.

- ¿Qué modelo de ramificación recomendaría y por qué? Justifique su respuesta basándose en el control y la estabilidad.
  GitFlow. Al tratarse de un equipo grande, jerárquico, con auditorías estrictas y lanzamientos trimestrales, GitFlow provee el control defensivo y las etapas de aprobación necesarias a través de sus ramas estables y de liberación (release/hotfix).

3. Feature Flags (Interruptores de Código): En equipos que integran cambios diariamente a la rama principal, ¿cómo permiten las Feature Flags que un programador suba código de una funcionalidad incompleta sin afectar la experiencia del usuario final?

Permiten envolver el código nuevo dentro de una condición lógica que se mantiene apagada en producción. De este modo, el código se integra continuamente a la rama principal sin afectar ni exponer la funcionalidad incompleta al usuario final.

Parte 2: Procesos de Integración Colaborativa

4. El Pull Request (PR) como Herramienta de Comunicación: Un PR no es solo un comando técnico, es una petición formal. Mencione qué elementos de información son indispensables en una descripción de PR para que un compañero pueda entender el cambio sin tener que leer línea por línea el código inicialmente.

Un PR profesional debe incluir: un título claro que resuma el cambio, una descripción detallada explicando el qué, cómo y por qué, un checklist de validación (tests, linters), y evidencias visuales (logs o capturas de pantalla).

5. La Cultura de la Code Review: ¿Cuál es el propósito de una revisión de código más allá de encontrar errores de sintaxis? Explique la importancia de la transferencia de conocimiento y el cumplimiento de principios como DRY (Don't Repeat Yourself) y SOLID.

Su propósito va más allá de buscar errores de sintaxis; busca asegurar la mantenibilidad, seguridad y calidad arquitectónica, fomentando la transferencia de conocimiento técnico y el cumplimiento de principios como DRY y SOLID.

6. Estrategias de Merge y el Historial: Compare las siguientes tres formas de integrar una rama y explique qué impacto tienen en la estética del grafo de Git:

- Create a Merge Commit (Historial verídico): Mantiene todos los commits de la rama y los une con un commit de fusión. Prioriza la trazabilidad total, aunque ensucia el grafo.
- Squash and Merge (Historial limpio): Comprime todos los commits de la funcionalidad en uno solo antes de unirlo. Deja un historial sumamente limpio y legible.
- Rebase and Merge (Historial lineal): Mueve los commits al frente de la rama principal de forma lineal sin commit de fusión. Crea una línea de tiempo perfectamente recta, pero reescribe el historial.

Parte 3: Estandarización y Documentación del Proyecto

7. Guías de Contribución (CONTRIBUTING.md): Usted es el líder de un proyecto y llega un nuevo integrante. ¿Qué información específica debería encontrar este desarrollador en el archivo CONTRIBUTING.md para empezar a codificar siguiendo las reglas del equipo?

Debe incluir los pasos para la configuración del entorno local y dependencias, reglas de estilo de código (linters, formato), convenciones para el nombramiento de ramas, y los requisitos mínimos obligatorios para aceptar un Pull Request.

8. Architecture Decision Records (ADRs): Explique la utilidad de un ADR. ¿Cómo ayuda este documento a que un desarrollador que se suma al proyecto un año después entienda por qué se tomó una decisión técnica específica en el pasado?

Un ADR registra formalmente una decisión técnica clave, el contexto que la motivó, la solución adoptada y sus consecuencias. Ayuda a que un nuevo integrante comprenda las razones detrás de dichas elecciones arquitectónicas un año después sin necesidad de reabrir discusiones pasadas.

9. Plantillas de Trabajo (Templates): ¿Qué beneficio aporta el uso de plantillas para Issues y Pull Requests en la organización diaria de un equipo de desarrollo?

Estandarizan los reportes de errores (asegurando pasos de reproducción y contextos) y los Pull Requests (garantizando explicaciones claras y validaciones previas), optimizando la organización diaria del equipo.

Parte 4: Validación de Estándares Locales

10. Git Hooks y la "Primera Línea de Defensa": Explique qué es un Git Hook (como Husky) y cómo ayuda a que el equipo mantenga un estándar de código (Linting) antes de que los cambios siquiera lleguen al repositorio compartido. ¿Por qué es importante validar el código en la máquina del desarrollador?

Un Git Hook es un script automatizado que se ejecuta en eventos locales de Git (como el pre-commit) para verificar linters, formateadores y pruebas unitarias. Es vital validarlo en la máquina del desarrollador para aplicar la filosofía Fail-Fast localmente, evitando subir código defectuoso al repositorio compartido y ahorrando recursos del servidor.